

ORPHE CORE SDK for Android

ORPHE COREに接続するためのJava SDKを提供します。

インストール

aarファイルを追加

※開発途中でありaarファイルが最新でない場合があります。その場合は上記のソースそのものを利用するでSDKを追加してください。

1. **orphecoresdk-release-xxx.aar**を下記のURLに従いインポートします。

https://zenn.dev/apple_nktn/articles/d6f7e0cac8d413

サンプルアプリの使い方

サンプルアプリでは、ORPHE INSOLEをスキャンして接続し、センサー値を計測できます。計測結果はアプリ内の履歴へ保存され、左右のデータをCSVファイルとして端末へ保存または他のアプリへ共有できます。

1. スキャンする

1. 端末のBluetoothを有効にしてサンプルアプリを起動します。
2. 初回起動時にBluetoothのスキャン権限を求められた場合は許可します。Android 11以前では位置情報の権限も必要です。
3. ORPHE INSOLEのリセットボタンを押して光らせ、アドバタイズ状態にします。
4. **Scan Devices**を押します。
5. LeftまたはRightの欄にデバイスIDと充電状態が表示されるまで待ちます。見つからない場合は**Scan Again**を押して再試行します。

2. 接続する

1. 接続する側の**Left Connect**または**Right Connect**を押します。
2. 同じ側に複数のデバイスが見つかった場合は、表示されたデバイスIDと充電状態を確認して接続先を選択します。
3. 「機器に接続されました」と表示されたことを確認します。

左右両方を接続することも、片側だけを接続して計測することもできます。接続を解除する場合は**Left Disconnect**または**Right Disconnect**を押します。

3. 計測する

1. 必要に応じて、画面上部で受信方式とサンプリングレートを選択します。
 - **Realtime**: デバイスからリアルタイムにセンサー値を受信します。100Hzまたは200Hzを選択できます。
 - **FIFO**: 欠損したデータを可能な限り回収しながら受信します。サンプリングレートは200Hzに固定されます。

- QuaternionグラフはRealtimeかつ100Hzを選択した場合のみ表示されます。デバイス側（0x38）が返すクオータニオンのみ動作検証済みのため、他の組み合わせでは表示しません。
- 2. Startを押して計測を開始します。
- 3. センサー値のグラフと、Measuring / Left: ... / Right: ...に表示される左右の受信件数を確認します。
- 4. 計測を終了する場合はStop & Saveを押します。計測が停止し、受信したデータがMeasurement Historyへ保存されます。

Clearは、画面上のグラフと現在の計測データを消去します。すでにMeasurement Historyへ保存された結果は削除されません。

4. 結果をCSVファイルとして保存する

Stop & Saveでは計測結果がアプリ内のMeasurement Historyへ保存されます。端末上の任意の場所へCSVファイルとして書き出す場合は、次の操作を行います。

1. Measurement Historyから対象の結果にあるDetailを押します。
2. 計測日時、計測時間、左右の受信件数、受信方式、サンプリングレート、CSVのプレビューを確認します。
3. Save Left CSVまたはSave Right CSVを押します。
4. Androidのファイル保存画面で保存先とファイル名を指定します。

データが存在する側だけ保存ボタンが有効になります。履歴そのものを削除する場合はDeleteを押します。

CSVのクオータニオン列（quatW~quatZ）は、Realtimeかつ100Hzで計測した場合だけ出力されます。Request / FIFO、およびRealtimeの200Hzでは、これら4列はヘッダーを含めて出力されません。Euler角は全モードでCSVへ出力されません。

5. 結果をシェアする

1. Measurement Historyにある対象結果のShareを押します。詳細画面のShare CSVからも同じ操作ができます。
2. Androidの共有先選択画面から、CSVファイルを渡すアプリを選択します。

片側だけを計測した場合は1つ、左右両方を計測した場合は左右のCSVファイル2つが共有されます。

圧力補正値を設定する

- 左右のインソールへ接続すると、それぞれのPressure Calibrationボタンが有効になります。
- 6点の圧力センサーごとにcoefficient1、coefficient2、coefficient3、thresholdを入力し、Saveを押すと受信中の圧力値へ即時反映されます。
- 設定はデバイスID別に端末へ保存され、同じデバイスへ再接続したときに自動で適用されます。
- Reset to defaultsを押すと、そのデバイスの保存値を削除し、全6点を既定値へ戻します。
- サンプルアプリは補正値を自動推定しません。測定などで決定した補正値を手動入力するための機能です。

利用方法

事前準備

※`app/src/main/java/io/orphe/orphecoresdkforandroid/MainActivity.java`にサンプルコードが記載されています。

- SDKモジュールのManifestには権限宣言が含まれていないため、利用側アプリの `AndroidManifest.xml`へ以下を追加します。

```
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission
  android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission
  android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

- パーミッションの確認と必要であればリクエストを行います。Androidのバージョンによって位置情報の権限が必要な場合とBluetooth系の権限が必要な場合があります。

```
private boolean hasBlePermissions() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        return checkSelfPermission(Manifest.permission.BLUETOOTH_SCAN)
            == PackageManager.PERMISSION_GRANTED
            &&
            checkSelfPermission(Manifest.permission.BLUETOOTH_CONNECT)
                == PackageManager.PERMISSION_GRANTED;
    }
    return
        checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION)
            == PackageManager.PERMISSION_GRANTED;
}

private void requestBlePermissions() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        requestPermissions(new String[]{
            Manifest.permission.BLUETOOTH_CONNECT,
            Manifest.permission.BLUETOOTH_SCAN}, 0);
    } else {
        requestPermissions(new String[]{
            Manifest.permission.ACCESS_FINE_LOCATION,
            Manifest.permission.BLUETOOTH_ADMIN,
            Manifest.permission.BLUETOOTH}, 0);
    }
}
```

ORPHE INSOLEの場合

- `OrpheInsoleCallback`オブジェクトを作成します。この中にORPHE INSOLEの各イベントに対しての動作を記述します。

```
private final OrpheInsoleCallback mOrpheInsoleCallback = new
OrpheInsoleCallback() {
    @Override
    public void gotInsoleValues(OrpheInsoleValue[] values) {
        Log.d(TAG, values[0].toString());
    }

    @Override
    public void gotDeviceInfo(DeviceInfoValue value) {
        Log.d(TAG, value.batteryStatus.name());
    }

    @SuppressWarnings("MissingPermission")
    @Override
    public void onScan(BluetoothDevice bluetoothDevice,
OrpheScannedMeta meta) {
        if (mConnectionStatusTextViewLeft != null) {
            if (bluetoothDevice != null) {
                final String deviceId = meta.deviceId;
                Log.d(TAG, String.format("%s : 機器が見つかりました",
deviceId));
            } else {
                Log.d(TAG, "機器が見つかりませんでした");
            }
        }
    }

    @SuppressWarnings("MissingPermission")
    @Override
    public void onConnect(BluetoothDevice bluetoothDevice) {
        if (mConnectionStatusTextViewLeft != null) {
            Log.d(TAG, String.format("%s : 機器に接続されました",
bluetoothDevice.getName()));
        }
    }

    @SuppressWarnings("MissingPermission")
    @Override
    public void onDisconnect(BluetoothDevice bluetoothDevice) {
        if (mConnectionStatusTextViewLeft != null) {
            Log.d(TAG, String.format("%s : 機器の接続が解除されました",
bluetoothDevice.getName()));
        }
    }
};
```

- 作成したOrpheInsoleCallbackと対応する取り付け位置（OrpheSidePosition.leftPlantarなど）を指定してOrpheInsoleオブジェクトを作成します。

```
mOrpheInsole = new OrpheInsole(this, mOrpheInsoleCallbackLeft,
OrpheSidePosition.leftPlantar);
```

- 作成した`OrpheInsole`オブジェクトの`startScan`を呼び出します。

```
mOrpheInsole.startScan();
```

- ORPHE INSOLEのリセットボタンを押し光らせます。
 - 光った場合はアドバタイズしている状態になります。
- `OrpheInsoleCallback`の`onScan`に見つかったアドバタイズ中のORPHE INSOLEの`BluetoothDevice`オブジェクトが渡されます。
- `OrpheInsoleCallback`の`onScan`で渡された`BluetoothDevice`を`OrpheInsole`オブジェクトの`connect`に渡すことで接続されます。

```
mOrpheInsole.connect(mBluetoothDevice);
```

- `OrpheScannedMeta`の`deviceId`を参照することでデバイスIDを取得できます。
 - `OrpheScannedMeta`の`chargeStatus`を参照することで充電の状況（Wireless or Wired）。
- 接続後`OrpheInsoleCallback`の`onConnect`のコールバックが呼び出されます。
- センサー値の取得に関して
 - `OrpheInsoleSensorConfig`を指定すると、接続後の初期設定として受信方式とサンプリングレートを選択できます。

```
OrpheInsoleSensorConfig config = new OrpheInsoleSensorConfig(
    OrpheSensorReceiveMode.realtime,
    OrpheInsoleSamplingRate.hz200
);
mOrpheInsole = new OrpheInsole(
    this,
    mOrpheInsoleCallback,
    OrpheSidePosition.leftPlantar,
    OrpheAccRange.range16,
    OrpheGyroRange.range2000,
    false,
    config
);
```

- `OrpheSensorReceiveMode.realtime`: デバイスが送信したセンサー値をそのままNotifyで受け取ります。

- **OrpheSensorReceiveMode.request**: アプリから指定した範囲だけを手動でリクエストします。
- **OrpheSensorReceiveMode.fifo**: SDKがcarry-over方式で常時リクエストし、欠損検出・再要求・重複排除・全体値へのシリアル順マージまで自動で行います。受信値のコールバックは欠損回収を待たず即時に呼ばれます。
- **OrpheInsoleSamplingRate.hz100**: 100Hz出力。**realtime**ではクォータニオン付きの0x38を受信し、SDKがオイラー角を計算します。**request / fifo**ではFWの200Hz蓄積データ（0x36）をSDK内で姿勢計算してから100Hzへ間引きます（未検証のため動作は保証しません）。
- **OrpheInsoleSamplingRate.hz200**: 200Hz出力。**realtime**ではクォータニオンを含まない0x37のため、クォータニオン・オイラー角は0です。**request / fifo**ではFWの蓄積・再要求形式である0x36を受信し、SDK内でクォータニオンとオイラー角を計算します（未検証のため動作は保証しません）。

request / fifoはAndroid 13（API 33）以上で利用できます。Android 8.0（API 26）からAndroid 12L（API 32）までは**realtime**を使用してください。

SDK利用側で欠損を可能な限り回収しながら100Hz出力を得る場合は以下のように指定します。FWから受け取る0x36は200Hz固定で、SDKは4点すべてを5ms間隔で姿勢計算した後、0ms / 10msの2点へ間引きます。アプリ側に姿勢計算用タイマーや再要求処理は不要です。

```
OrpheInsoleSensorConfig config = new OrpheInsoleSensorConfig(
    OrpheSensorReceiveMode.fifo,
    OrpheInsoleSamplingRate.hz100
);
```

FIFOの既定値は、200msごとに最大200シリアルを確認し、応答を最大5秒待ちます。Android版では応答を1件以上受信した後に250ms無通信となった時点で要求を終了し、未着値を次回へcarry-overするため、部分的なBLEロスのたびに5秒待ち続けません。FWから**noData**が返った値だけを回復不能として**OrpheInsoleCallback.sensorValueIsNotFound**へ通知します。carry-overが100シリアルを超えた場合、または最新値との差が1,500シリアルを超えた場合は、FWリングバッファの安全範囲へ再同期します。

FIFOでは、値を受信するたびに**OrpheInsoleValueUpdate**版のコールバックが呼ばれます。**getDeltaValues()**は今回受信した差分、**getAllValues()**はその時点までにSDKが取得・欠損回収できた全値です。欠損パケットが後から回収された場合は、その位置から後続のクォータニオンを時系列順に再計算します。全値はシリアル順にマージされ、実際に**getAllValues()**を呼んだ時だけ配列が生成されます。

```
OrpheInsoleCallback callback = new OrpheInsoleCallback() {
    @Override
    public void gotInsoleValues(OrpheInsoleValueUpdate update) {
        OrpheInsoleValue[] deltaValues = update.getDeltaValues();
        OrpheInsoleValue[] allValues = update.getAllValues();
    }
};
```

従来の`gotInsoleValues(OrpheInsoleValue[] values)`を実装している場合も変更は不要です。FIFOでは同メソッドへ即時差分が渡されます。現在の全値だけを後から参照する場合は`OrpheInsole#getFifoValues()`も利用できます。

- `OrpheSensorReceiveMode.request`で手動取得する場合は、接続後にセンサー値送信のリクエストを送ります。1範囲なら`requestInsoleValue(startSerialNumber, length)`、複数範囲なら`requestInsoleValue(OrpheValueRequest[])`を使用できます（最大30範囲）。

```
mOrpheInsole.requestInsoleValue(1200, 100);
```

- `requestLatestInsoleValue`を呼び出すことで最新のセンサー値を取得することが可能です。

```
mOrpheInsole.requestLatestInsoleValue();  
// もしくは  
mOrpheInsole.requestLatestInsoleValue(100);
```

- `length`のパラメーターを指定した場合は、**センサー値の最終取得時刻から予想されるシリアル番号からlength件**を取得します。
 - 取得件数が少なすぎると時間のズレ等でうまく取得できない可能性があるもので100件程度までは数を増やしてください。
- `length`のパラメータを指定しない場合は、**センサー値の最終取得時刻から予想されるシリアル番号から現在時刻までのセンサー値**を取得します。
- 最新シリアル番号をまだ取得していない場合、最初の呼び出しでは現在のシリアル番号だけを要求して終了します。
`OrpheInsoleCallback.gotCurrentSerialNumber`が呼ばれた後に、もう一度呼び出してください。
- `requestInsoleValue`を呼び出すことで自由にデバイス内に一時保存されているセンサー値を取得することができます。
 - `OrpheValueRequest`に最初のシリアル番号とそこから取得する件数を指定してパラメータに渡してください。（最大30種類リクエストを送ることが可能）
- 新しい手動リクエストはデバイス上の古いリクエストを置き換えます。常時取得には、要求を直列化する`fifo`を使用してください。
- リクエストされたセンサー値は200Hz形式の0x36としてNotifyで送信され、SDKがクオータニオンを計算してから`OrpheInsoleCallback`の`gotInsoleValues`に渡します。FIFOでは欠損箇所以降も待機させず、受信できた値から即時に通知し、欠損回収後にSDK内の後続値を再計算します。
 - `hz200`では、6点の圧力センサー値を含むセンサー値を1シリアル番号につき4時点分取得します。`hz100`ではSDKがその4時点を2時点へ間引きます。どちらもシリアル番号1増えるごとのインターバルは理論上20msです。

- Euler角の計算と公開フィールドは一時的に無効化されています。
 - Madgwick 6軸フィルタは磁気センサーを使わないため、算出されるクオータニオンの yaw 方向には時間経過によるドリフトがあります。
 - `startTime`、`endTime`、`receivedAt`はepochミリ秒です。秒へ変換する場合は`1000.0`で割ります。
- 作成した`OrpheInsole`オブジェクトの`disconnect`を呼び出すことで切断します。

```
mOrpheInsole.disconnect();
```

- 作成した`OrpheInsole`オブジェクトの`status`で現在の接続ステータスを把握することが可能です。
- 作成した`OrpheInsole`オブジェクトの`getDeviceInfo`を呼び出すことでバッテリー情報を含む ORPHE INSOLE の情報を取得することができます。取得した値は`OrpheInsoleCallback`の `gotDeviceInfo`に渡されます。

```
mOrpheInsole.getDeviceInfo();
```

- リアルタイムモードへの切り替えは`setSensorRequestMode`を呼び出すことで可能です。
 - リセットされるとともに戻るため接続時に 1 度だけ実行することを推奨します。
 - `OrpheInsoleSensorConfig`を指定した場合はSDKがNotify開始後に自動で切り替えます。
 - インソール（圧力込み）用の200Hzリアルタイムモードは `OrpheSensorRequestMode.realtimeForInsole`、100Hzリアルタイムモードは `OrpheSensorRequestMode.realtimeForInsoleWithQuaternion`と指定します。

```
mOrpheInsole.setSensorRequestMode(OrpheSensorRequestMode.realtimeForInsole);
```

- 圧力の変換係数は`setPressureCalibration`メソッドで6点分を一括設定できます。各部位に `coefficient1`、`coefficient2`、`coefficient3`、`threshold`を設定できます。

```
OrpheInsolePressureCalibration calibration = new
OrpheInsolePressureCalibration(
    // toeInside
    new OrpheInsolePressureCoefficient(2.77942, 0.00235, 4.14411,
240.0),
    // midInside
    new OrpheInsolePressureCoefficient(2.70000, 0.00230, 4.10000,
235.0),
    // toeOutside
    new OrpheInsolePressureCoefficient(2.80000, 0.00240, 4.20000,
245.0),
    // center
    new OrpheInsolePressureCoefficient(2.60000, 0.00220, 4.00000,
```



```

230.0),
    // midOutside
    new OrpheInsolePressureCoefficient(2.90000, 0.00250, 4.30000,
250.0),
    // heel
    new OrpheInsolePressureCoefficient(3.00000, 0.00260, 4.40000,
255.0)
);
mOrpheInsole.setPressureCalibration(calibration);

```

- `OrpheInsolePressureCoefficient`の4引数は順に`coefficient1`、`coefficient2`、`coefficient3`、`threshold`です。
- 従来の2引数コンストラクタも利用でき、その場合は`coefficient2 = 0.00235`、`threshold = 240mV`が補完されます。
- コンストラクタの部位順は`toeInside`、`midInside`、`toeOutside`、`center`、`midOutside`、`heel`です。
- 既定値へ戻す場合は
`setPressureCalibration(OrpheInsolePressureCalibration.DEFAULT)`を使用します。
- 従来の`setCoefficient(sensorPosition, coefficient, value)`も引き続き利用でき、現在の一括設定のうち指定した1項目だけを更新します。
- 計算式は`coefficient1 * exp(coefficient2 * milliVolt) + coefficient3`です。
- `milliVolt <= threshold`の場合は`0N`として扱います。
- 閾値以下、負の計算結果、または異常な入力値は`0N`として扱います。

ORPHE COREの場合

- `OrpheCoreCallback`オブジェクトを作成します。この中にORPHE COREの各イベントに対しての動作を記述します。

```

private final OrpheCoreCallback mOrpheCoreCallback = new
OrpheCoreCallback() {
    @Override
    public void gotSensorValues(OrpheSensorValue[] values) {
        Log.d(TAG, values[0].toString());
    }

    @Override
    public void gotDeviceInfo(DeviceInfoValue value) {
        Log.d(TAG, value.batteryStatus.name());
    }

    @SuppressWarnings("MissingPermission")
    @Override
    public void onScan(BluetoothDevice bluetoothDevice,
OrpheScannedMeta meta) {
        if (mConnectionStatusTextViewLeft != null) {
            if (bluetoothDevice != null) {
                final String deviceId = meta.deviceId;

```

```

        Log.d(TAG, String.format("%s : 機器が見つかりました",
deviceId));
    } else {
        Log.d(TAG, "機器が見つかりませんでした");
    }
}

@Override
public void onConnect(BluetoothDevice bluetoothDevice) {
    if (mConnectionStatusTextViewLeft != null) {
        Log.d(TAG, String.format("%s : 機器に接続されました",
bluetoothDevice.getName()));
    }
}

@Override
public void onDisconnect(BluetoothDevice bluetoothDevice) {
    if (mConnectionStatusTextViewLeft != null) {
        Log.d(TAG, String.format("%s : 機器の接続が解除されました",
bluetoothDevice.getName()));
    }
}
};

```

- 作成した`OrpheCoreCallback`と対応する取り付け位置（`OrpheSidePosition.leftPlantar`など）を指定して`Orphe`オブジェクトを作成します。

```

mOrphe = new Orphe(this, mOrpheCoreCallbackLeft,
OrpheSidePosition.leftPlantar);

```

- 作成した`Orphe`オブジェクトの`startScan`を呼び出します。

```

mOrphe.startScan();

```

- ORPHE COREを振り光らせます。
 - 光った場合はアドバタイズしている状態になります。
- `OrpheCoreCallback`の`onScan`に見つかったアドバタイズ中のORPHE COREの`BluetoothDevice`オブジェクトが渡されます。
- `OrpheCoreCallback`の`onScan`で渡された`BluetoothDevice`を`Orphe`オブジェクトの`connect`に渡すことで接続されます。

```
mOrphe.connect(mBluetoothDevice);
```

- **OrpheScannedMeta**の**deviceId**を参照することでデバイスIDを取得できます。
- 接続後**OrpheCoreCallback**の**onConnect**のコールバックが呼び出されます。
- センサー値の取得に関して
 - **OrpheCoreSensorConfig**でINSOLEと同じ3つの受信方式を指定できます。

```
OrpheCoreSensorConfig config = new OrpheCoreSensorConfig(
    OrpheSensorReceiveMode.fifo
);
mOrphe = new Orphe(
    this,
    mOrpheCoreCallback,
    OrpheSidePosition.leftInstep,
    config
);
```

- **realtime**: リアルタイムNotify
- **request**: アプリが指定した範囲だけを手動取得
- **fifo**: SDKがcarry-over方式で継続取得と欠損回収を自動実行

request / **fifo**はAndroid 13（API 33）以上で利用できます。Android 8.0（API 26）からAndroid 12L（API 32）までは**realtime**を使用してください。

- **request**で手動取得する場合は、接続後にリクエストを送ります。1範囲なら**requestSensorValue(startSerialNumber, length)**、複数範囲なら**requestSensorValue(OrpheValueRequest[])**を使用できます（最大30範囲）。

```
mOrphe.requestSensorValue(1200, 100);
```

- **requestLatestSensorValue**を呼び出すことで最新のセンサー値を取得することが可能です。

```
mOrphe.requestLatestSensorValue();
// もしくは
mOrphe.requestLatestSensorValue(100);
```

- **requestLatestSensorValue()**は**requestLatestSensorValue(100)**を呼び出し、100シリアル分を取得します。
- **length**のパラメーターを指定した場合は、**センサー値の最終取得時刻から予想されるシリアル番号からlength件**を取得します。

- 取得件数が少なすぎると時間のズレ等でうまく取得できない可能性があるの
で100件程度までは数を増やしてください。
 - 最新シリアル番号をまだ取得していない場合、最初の呼び出しでは現在のシリアル
番号だけを要求して終了します。
`OrpheCoreCallback.getCurrentSerialNumber`が呼ばれた後に、もう一度呼
び出してください。
 - `requestSensorValue`を呼び出すことで自由にデバイス内に一時保存されているセンサ
ー値を取得することができます。
 - `OrpheValueRequest`に最初のシリアル番号とそこから取得する件数を指定してパ
ラメータに渡してください。(最大30種類リクエストを送ることが可能)
 - 新しい手動リクエストはデバイス上の古いリクエストを置き換えます。常時取得には、要
求を直列化する`fifo`を使用してください。
- リクエストされたセンサー値はNotifyで送信され、SDKがクオータニオンと重力成分を計算して
から`OrpheCoreCallback`の`gotSensorValues`に渡されます。
 - Euler角の計算と公開フィールドは一時的に無効化されています。
 - 200Hzでは、加速度とジャイロを含むIMUセンサー値を1シリアル番号につき8時点分取得
します。シリアル番号1増えるごとのインターバルは理論上40msです。現行実装では
Request / FIFOで取得した8時点の`startTime` / `endTime`は5ms間隔にならないため、時
点間隔の算出には使用しないでください。
 - `startTime`、`endTime`、`receivedAt`はepochミリ秒です。秒へ変換する場合は`1000.0`
で割ります。
- `fifo`の既定値はCORE / INSOLE共通で、200msごと・最大200シリアル・応答待ち5秒・carry-
over上限100シリアル・リングバッファ安全上限1,500シリアルです。Android版は応答を1件以上
受信した後の無通信が250ms続くと早期に要求を終了します。BLEタイムアウトは次回要求へ持
ち越し、デバイス内に対象シリアルが残っておらずFWから`noData`が返った場合だけ、
`OrpheCoreCallback.sensorValueIsNotFound` (INSOLEでは
`OrpheInsoleCallback.sensorValueIsNotFound`) が番号ごとに呼ばれます。
- COREのFIFOでもINSOLEと同様に、`OrpheSensorValueUpdate`から今回差分と再計算済み全
値を取得できます。既存の配列版コールバックは変更不要です。

```
OrpheCoreCallback callback = new OrpheCoreCallback() {
    @Override
    public void gotSensorValues(OrpheSensorValueUpdate update) {
        OrpheSensorValue[] deltaValues = update.getDeltaValues();
        OrpheSensorValue[] allValues = update.getAllValues();
    }
};

OrpheSensorValue[] currentValues = mOrphe.getFifoValues();
```

- SDK内の姿勢計算は`orphe_insole`と同じMadgwick 6軸フィルタ (初期姿勢`[w,x,y,z] = [1,0,0,0]`、`beta = 0.1`、`dt = 5ms`) を使用します。加速度・ジャイロのX/Y/ZはBLE復号

値を並べ替え・符号反転せず使用し、ジャイロだけdpsからrad/sへ変換します。磁気センサーを使わないため、roll/pitchは重力で補正されますがyawには時間経過によるドリフトがあります。

- またセンサー値のNotifyが有効になり、**OrpheCoreCallback**の**gotSensorValues**に各Notifyごとで送信されたセンサー値が渡されます。（1度のNotifyで最大4つのセンサー値が渡されます）
 - Notifyは50Hzで送られており4つのセンサー値を送ることで最大200Hzのセンサー値を取得することができます。
 - **startTime**、**endTime**、**receivedAt**はepochミリ秒です。秒へ変換する場合は**1000.0**で割ります。
- 作成した**Orphe**オブジェクトの**disconnect**を呼び出すことで切断します。

```
mOrphe.disconnect();
```

- 作成した**Orphe**オブジェクトの**status**で現在の接続ステータスを把握することが可能です。
- 作成した**Orphe**オブジェクトの**getDeviceInfo**を呼び出すことでバッテリー情報を含むORPHE COREの情報を取得することができます。取得した値は**OrpheCoreCallback**の**gotDeviceInfo**に渡されます。

```
mOrphe.getDeviceInfo();
```